Microsoft

Azure Key Vault Managed HSM

Migrating Azure Storage to HSM-Backed Customer Managed Keys

Author: Naresh Gupta (Senior Consultant)

This guide outlines a backup, restore, and disaster recovery strategy for **Azure Key Vault Managed HSM** in **GCC High, GCC and Commercial** environments, offering both conceptual guidance and a practical runbook to safeguard Microsoft 365 Customer Key assets and ensure compliance.

Overview

Organizations in regulated sectors — government agencies operating under FedRAMP High and DoD IL4 mandates, healthcare providers handling protected health information, and enterprise IT teams safeguarding sensitive intellectual property — must ensure that cloud-hosted data is encrypted with keys under their **exclusive control**. Azure Storage encrypts all data at rest by default with Microsoft-managed keys, but compliance regimes such as **FedRAMP High** and **DoD IL4** frequently require **customer-managed keys (CMKs)** stored in **FIPS 140-2 Level 3 validated** Hardware Security Modules. This white paper describes the **Contoso** approach to migrating Azure Storage Accounts from platform-managed encryption to **HSM-backed CMKs stored in Azure Key Vault Managed HSM**, using an automated PowerShell script (**migrate-sa.ps1**) purpose-built for **GCC High (Azure Government)** and **Commercial** environments.

The migration process is **automated, seamless, and non-disruptive**. Azure Storage wraps the root data encryption key with the customer-managed key, and **data always remains encrypted** throughout the transition — no downtime or additional action is required. Each storage account receives a **dedicated RSA-HSM key** in the Managed HSM, accessed through a **user-assigned managed identity**, ensuring that only identities explicitly authorized by the organization can decrypt data at rest.

Full Key Ownership & Compliance

By migrating to HSM-backed CMKs, Contoso ensures **exclusive ownership of encryption keys** with **FIPS 140-2 Level 3** hardware protection, satisfying requirements for handling sensitive data such as **ITAR export-controlled** or **CUI data** in GCC High environments.

Automated & Seamless Migration

A **PowerShell script** (`migrate-sa.ps1`) processes a CSV-driven list of storage accounts, creating or reusing HSM keys and configuring encryption with **no service interruption**. Protection by customer-managed keys takes effect **immediately**.

Enhanced Data Security

Each storage account receives a **unique 3072-bit RSA-HSM key** (non-exportable) generated in the Managed HSM. A **user-assigned managed identity** authenticates the storage account to the HSM, ensuring only authorized identities can access the key for wrap/unwrap operations.

Purpose and Context

Why Customer-Managed HSM Keys?

The **migrate-sa.ps1** PowerShell script automates the migration of Azure Storage Accounts to use **customer-managed encryption keys** backed by an **Azure Key Vault Managed HSM**. This switches storage account encryption from Microsoft-managed keys to keys that the organization controls exclusively in the Managed HSM. In **GCC High** environments, this is

often mandated for compliance with **stringent regulations** (e.g. FedRAMP High, DoD IL4), since Managed HSM provides **FIPS 140-2 Level 3 validated key protection** and exclusive key ownership by the customer. The Managed HSM is also characterized in the document's compliance section as a **single-tenant FIPS 140-3 Level 3 HSM service**, meeting the most current security standards. By using HSM-backed keys, the organization maintains full control over encryption keys, satisfying requirements for handling sensitive data such as **ITAR export-controlled** or **CUI data** in GCC High.

Azure Storage encryption with CMK is supported in GCC High for **blobs and files** automatically, and optionally for **Queue and Table storage** if enabled at account creation time. Blob storage and Azure Files are always protected by customer-managed keys once CMKs are configured for the storage account.

Even the cloud provider **cannot access the customer's keys** — because the keys reside in the organization's own Managed HSM, Microsoft's role is limited to managing the HSM service and storage infrastructure without access to the encryption keys, aligning with **Zero Trust principles**.

While developed for **Contoso's** GCC High deployment, the script and methodology apply equally to **federal agencies, healthcare organizations, and enterprise IT teams** operating in Azure Commercial or Azure Government clouds.

Azure Services Involved

The migration script interfaces with four principal Azure components. **Table 1** below summarizes each service's role in the solution.

Table 1: Key Azure Services & Resources in Migration

Azure Service / Resource	Role in HSM-Backed Storage Encryption
Azure Key Vault Managed HSM	Hosts the customer-managed RSA keys used to encrypt storage accounts. Provides HSM-grade key generation and storage. The script uses Get-AzKeyVaultKey and Add-AzKeyVaultKey cmdlets targeting the HSM by name ($HsmName). Requires appropriate roles (e.g. *Managed HSM Crypto Officer*) to create and manage keys.
Azure Storage Account	The target resource being configured to use customer-managed encryption. The script updates each storage account's encryption settings via Set-AzStorageAccount to enable CMKs, requiring Azure RBAC permission such as Storage Account Contributor or Owner on that resource.
Azure AD / Managed Identity	A user-assigned managed identity acts as the security principal that the storage account uses to authenticate to the HSM. This identity needs key access permissions on the HSM — at minimum wrapKey, unwrapKey, get on the CMK, or the equivalent *Managed HSM Crypto Service Encryption User* role scoped to the key.
Azure Subscription Context	The script switches Azure context between subscriptions as needed — one subscription may host the managed identity and HSM, while another hosts the storage accounts. The operator must have access to all relevant subscriptions.

Azure RBAC Considerations

Two distinct actors require role assignments for this solution to function:

- **Script operator** (the person running the PowerShell): Must have rights to **create and read keys** in the Managed HSM (e.g. *HSM Crypto Officer* or *HSM Administrator* role on the HSM) **and** to update storage account settings (e.g. **Contributor/Owner** on the storage account's resource group). If the HSM and storage accounts span subscriptions, the operator needs access in both.
- **User-assigned managed identity**: Must be pre-created and granted the built-in ***Managed HSM Crypto Service Encryption User*** role scoped to the specific key(s) on the HSM. This role grants the necessary data-plane actions (**unwrapKey**, **wrapKey**, **get**) for the storage service to use the key at runtime. Microsoft recommends scoping the role assignment to the **level of the individual key** to grant the fewest possible privileges. Removing the identity or its role assignment would cause the storage account to **lose access to its encryption key**.

Accounts Per Script Run

All CSV Entries

The script iterates through every row in the input CSV, processing each storage account in sequence.

Encryption Key Size

3072-bit RSA-HSM

Keys are generated as non-exportable 3072-bit RSA-HSM keys. Azure Storage also supports 2048 and 4096 bits.

Migration Script Workflow

The **migrate-sa.ps1** script reads a CSV list of target storage accounts and performs four core operations per account:

1. Initialization and Configuration

The script accepts a single parameter — the **path to a CSV file** (param($list)) containing the list of storage accounts to migrate. On startup, it generates a **timestamp** (yyyyMMdd-HHmmss format) and creates two log files:

- **$ts-sa-migration-log.txt** — a general log capturing all actions taken.
- **$ts-sa-migration-errors.txt** — a log specifically for errors encountered.

The global error preference is set to **$Global:ErrorActionPreference = "Stop"**, causing any unhandled error to immediately invoke catch blocks rather than continuing in a bad state.

Several **hard-coded environment constants** define the target infrastructure:

Variable	Value	Purpose
$hsmurl	https://mhsm-prd-cr2-wus3.managedhsm.azure.net/	Full URL of the Managed HSM endpoint
$HsmName	mhsm-prd-cr2-wus3	HSM resource name (used by -HsmName parameter)

$miid	ui-storagehsm-prd-cr2	Name of the user-assigned managed identity
$miidrg	rg-mhsm-prd-cr2-wus3	Resource group of the managed identity
$miid_sub	CR2	Subscription identifier for the identity/HSM subscription
$KeyType	RSA-HSM	HSM-protected RSA key type (non-exportable)
$KeySize	3072	3072-bit RSA key size for compliance and performance balance
$KeyOps	Encrypt, Decrypt, Sign, Verify, WrapKey, UnwrapKey	Permitted cryptographic operations; WrapKey/UnwrapKey are critical for storage encryption

These values are specific to Contoso's environment; organizations adopting this approach should **adjust them to match their own HSM instance, identity, and subscription structure**.

2. HSM Key Check and Creation

For each storage account, the script constructs a **unique key name** following the convention **key-<StorageAccountName>-cr2-westus3** (encoding the environment/region) and then:

- Calls **Get-AzKeyVaultKey -HsmName $HsmName -Name $KeyName** to check if a key already exists.
- If **no key exists**: creates a new key using **Add-AzKeyVaultKey** with parameters -HsmName, -Name, -KeyType RSA-HSM, -Size 3072, and -KeyOps. The user running this command must have the ***Managed HSM Crypto Officer*** role on the HSM. The HSM generates a 3072-bit RSA key pair internally; the private key material **never leaves the HSM**.
- If **key already exists**: the script logs that the key will be **reused**, ensuring **idempotency** — re-running the script on the same accounts does not create duplicate keys.

Critical error behavior: If key creation fails (permissions issue, connectivity, etc.), the script prints **\[FAILED]**, logs the error, and **exits immediately** — it does not proceed to subsequent storage accounts. This prevents continuing with an incomplete key setup.

3. Managed Identity Retrieval

The script switches Azure context to the **managed identity's subscription** (Set-AzContext -Subscription $miid_sub) and calls **Get-AzUserAssignedIdentity -ResourceGroupName $miidrg -Name $miid** to retrieve the identity's **Azure Resource ID**. This resource ID (in the format */subscriptions/<SubID>/resourceGroups/<RG>/providers/Microsoft.ManagedIdentity/userAssignedIdentities/<Name>*) uniquely identifies the identity and is required to attach it to the storage account and specify it for key access.

If the identity cannot be found (e.g. name mismatch or insufficient permissions), the script **exits** with an error.

4. Storage Account Encryption Configuration

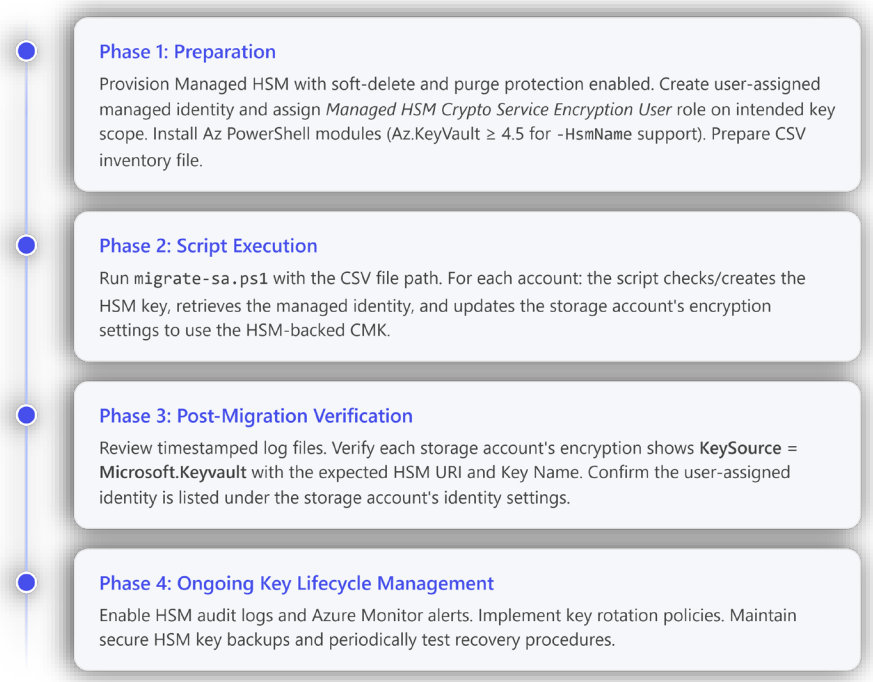
This is the culminating step. The script switches to the storage account's subscription and executes a single **Set-AzStorageAccount** command with the following parameters:

Parameter	Purpose
-ResourceGroupName $sarg -Name $saname	Identify the target storage account
-IdentityType UserAssigned	Enables user-assigned managed identity on the storage account
-UserAssignedIdentityId $mi_id	Attaches the specific managed identity to the storage account resource
-KeyVaultUri $hsmurl	Points to the Managed HSM's base URI as the key store
-KeyName $KeyName	Specifies which key in the HSM to use for encryption
-KeyVaultUserAssignedIdentityId $mi_id	Explicitly designates which identity authenticates to the HSM for key operations — removes ambiguity if multiple identities are attached

Automatic key version rotation: The script deliberately **omits the key version**, causing Azure Storage to use the **current active version** and **automatically roll to new versions** if the key is rotated in the future. This means subsequent key rotations do **not** require re-running the migration script.

Behind the scenes, Azure **immediately attempts to use the provided key** to wrap the storage account's Data Encryption Key (DEK). If the managed identity lacks the required wrap/unwrap permissions on the key, this command will fail with an authorization error.

Error handling differs from key creation: If the storage account update fails, the script logs the error but **continues to the next entry** in the CSV, allowing batch processing to complete even if individual accounts encounter issues. All failures are recorded in the error log for subsequent investigation.



Example Scenario: Contoso Migration

The script content includes a representative example:

- **Storage Account:** samhsmprdcr2wus3 in resource group rg-mhsm-prd-cr2-wus3, subscription ContosoGovSub1.
- **Managed HSM:** mhsm-prd-cr2-wus3.
- **Managed Identity:** ui-storagehsm-prd-cr2 in resource group rg-mhsm-prd-cr2-wus3 (subscription CR2).

When the script processes this account, it looks for key **key-samhsmprdcr2wus3-cr2-westus3** in the HSM. If not present, it creates a **3072-bit RSA-HSM key**. It switches to subscription CR2 to retrieve the identity's resource ID, then switches to the storage account's subscription and runs Set-AzStorageAccount to attach the identity and point encryption to the HSM key. The log output confirms success:

Key exists - will reuse! ... SA: HSM: Key: [OK]

Prerequisites and Setup

Before running the script, the following prerequisites must be satisfied:

- **Azure Environment Access:** The operator needs an Azure AD account with access to all relevant subscriptions (HSM/identity subscription and storage account subscriptions). In GCC High, connect PowerShell to **AzureUSGovernment**: Connect-AzAccount -Environment AzureUSGovernment. Verify context availability by running Get-AzContext or Get-AzSubscription to confirm subscriptions are visible.
- **Azure PowerShell Modules:** Install the **Az module** (which bundles all sub-modules), including: **Az.Accounts** (login/context), **Az.Storage** (storage account management), **Az.KeyVault** (HSM key operations), and **Az.Resources** / **Az.ManagedServiceIdentity** (managed identity commands). The script requires **Az.KeyVault version 4.4.0+** (and ideally **≥ 4.5**) to support the -HsmName parameter for Managed HSM cmdlets. Verify support by running Get-Help Add-AzKeyVaultKey and confirming -HsmName appears in the parameter list.
- **Managed HSM Instance:** A provisioned **Azure Key Vault Managed HSM** must be active and accessible in the target subscription. Confirm that **soft delete and purge protection are enabled** (Managed HSM enforces these by default, satisfying Azure Storage's requirement). The HSM endpoint URL must be reachable by Azure, and the HSM must reside in the **same Azure AD tenant** as the storage accounts (cross-tenant key usage is possible but requires additional setup).
- **User-Assigned Managed Identity:** Create the identity with the exact name, resource group, and subscription matching the script's configuration variables (e.g. ui-storagehsm-prd-cr2 in rg-mhsm-prd-cr2-wus3). **Assign HSM permissions** by granting identity the ***Managed HSM Crypto Service Encryption User*** role scoped to each key (recommended for least privilege) or scoped to the entire HSM (simpler for many accounts, but less granular — the identity gains access to **all keys** in the HSM). The role assignment uses the CLI command: az keyvault role assignment create --

hsm-name <HSMName> --role "Managed HSM Crypto Service Encryption User" --assignee <IdentityPrincipalId> --scope /keys/<KeyName>.

- **Storage Account Permissions:** The operator should have **Owner or Contributor** on the storage accounts or their resource groups to set encryption and attach identities. If not, permissions should be elevated before the migration window.
- **Dry Run Consideration:** The script has no built-in dry-run mode. However, operators can manually verify readiness by running Get-AzKeyVaultKey for expected key names and Get-AzStorageAccount to check current Key Source values before execution. The script is **largely idempotent** — reapplying to already-migrated accounts will detect existing keys, reuse them, and reapply settings with no adverse effects.

CSV Input File Format

Prepare a CSV file listing all targets storage accounts. **Table 3** details the required columns.

Table 3: Required Columns in Storage Accounts CSV Input File

Column Name	Description	Example Value
resourcename	The name of the Azure Storage Account (must be globally unique in Azure)	mygovstorageacct
resourcegroupname	The name of the Resource Group containing the Storage Account	GovProdResources
subscription	The subscription identifier or name where the Storage Account exists. In Azure Gov, subscription GUIDs can be used. The script uses this to set context via Set-AzContext	12345678-90ab-cdef-1234-567890abcdef or Contoso Gov Prod

Important: Using the subscription **GUID** is safest to avoid ambiguity. Ensure no extra whitespace or BOM characters at the start of the file, as these may confuse Import-Csv. Every storage account listed should share the same HSM and identity configuration; if some accounts require different HSMs or identities, multiple script runs with different parameters are needed.

Running the Script

Execute the script from PowerShell with the CSV path as argument.

Real-time console output shows each account being processed with color-coded status indicators — **green \[OK]** for success and **magenta \[FAILED]** for errors. A typical successful run for one account displays:

Checking for Encryption Key ... [OK] Key exists – will reuse! Switching to MI Subscription Getting Managed User-Assigned Identity ... [OK] Switching to SA Subscription SA: HSM: Key: Updating Storage Account to use CMK ... [OK]

Post-Run Verification

After completion, verify migration results through multiple channels:

- **Log files:** Open *-sa-migration-log.txt to review per-account outcomes. Inspect* -sa-migration-errors.txt (if non-empty) to identify which accounts failed and why.
- **PowerShell verification:** For each account, run (Get-AzStorageAccount -ResourceGroupName <RG> -Name <SA>).Encryption to confirm KeySource = Microsoft.Keyvault and that KeyVaultProperties shows the expected HSM key URI and version. The **Identity** blade should list the attached user-assigned identity.
- **Azure Portal:** Navigate to each storage account → **Encryption** blade to confirm **Customer Managed Keys** are in use with the Key URI referencing the HSM. Check the **Identity** blade for the user-assigned identity under "User-assigned identities".
- **HSM key verification:** Use Get-AzKeyVaultKey -HsmName <HSMName> -Name <KeyName> to confirm the key exists and check its access control for the identity's role assignment.
- **Functional test:** List storage account keys (Get-AzStorageAccountKey) or upload/download a blob to verify the storage service can successfully unwrap the encryption key through the configured identity.

Roles and Permissions Summary

Table 2 consolidates the required role assignments for each operation in the migration script.

Table 2: Roles, Actors, and Permissions per Migration Action

Script Operation	Azure Service / Resource	Actor	Required Role / Permission
Key lookup (Get-AzKeyVaultKey)	Azure Key Vault Managed HSM	Script operator (control plane)	*Managed HSM Crypto User* or *Crypto Officer* (to read key metadata)
Key creation (Add-AzKeyVaultKey)	Azure Key Vault Managed HSM	Script operator (control plane)	*Managed HSM Crypto Officer* role (to create a new key)
Retrieve managed identity (Get-AzUserAssignedIdentity)	Azure Resource Manager (Managed Identity)	Script operator (Azure RBAC)	Reader or Contributor on the identity's Resource Group (to read identity)
Attach identity to storage (Set-AzStorageAccount -IdentityType UserAssigned)	Storage Account	Script operator (Azure RBAC)	Owner/Contributor on Storage Account (to modify identity settings)
Configure CMK encryption (Set-AzStorageAccount -KeyVaultUri ...)	Storage Account & Key Vault	Script operator (control plane) and storage account's managed identity (data plane)	Owner/Contributor on Storage Account (to set encryption settings); *Managed HSM Crypto Service Encryption User* role assigned to the storage account's identity on the key (to wrap/unwrap DEK at runtime)

Note: In Azure Government, these roles have the **same names** as in Azure Commercial, but role assignments must be made within the **Azure Gov portal/PowerShell context** — they are separate from Azure Commercial role assignments.

Security Best Practices and Considerations

Migrating to customer-managed HSM keys **strengthens security** but introduces operational responsibilities that must be carefully managed.

- **Key Protection & Lifecycle: Never delete or disable** a key that is actively protecting a storage account — doing so renders **all data in that storage account unreadable**, as Azure Storage would be unable to decrypt the stored content. Managed HSM always enforces a **7-day soft-delete window** for keys; if someone deletes a key by mistake, it can be restored within that window before permanent purging. **Rotate keys periodically** by creating new key versions under the same key name — because the script does not pin a specific key version, Azure Storage will **automatically start using new versions** without requiring a script re-run. Maintain **secure backups** of HSM keys and the HSM's security domain, especially in GCC High where high-value data is at stake.
- **Managed Identity Security:** The user-assigned identity holds powerful access — it can **unwrap the storage account's encryption key**. Only assign this identity to **storage accounts** and do not attach it to VMs, App Services, or other resources that could be compromised. **Monitor identity usage** through Azure Key Vault audit logs (AuditEvent diagnostic category): only the storage account's internally managed requests should be using the identity. Anomalous access may indicate a breach. If the identity is compromised, remove its role assignment on the HSM or detach it from the storage account to prevent unauthorized decryption.
- **Least Privilege:** The identity is granted only **wrap/unwrap/get** permissions — no ability to create or delete keys. Administrators running the script should use privileged accounts (Crypto Officer, Contributor) **only for the migration event**, then transfer ongoing monitoring to less privileged read-only roles. The solution uses **one shared identity for all storage accounts** for simplicity; this creates a single point of access. Organizations requiring stronger isolation can use **one identity per storage account** at the cost of increased management overhead for identities and role assignments.
- **Logging & Monitoring:** The script's own log files contain resource names and Azure error output; treat them as **internal information** and secure them appropriately. Enable **Azure Monitor logs** for both storage accounts and the Managed HSM. The HSM's audit log records all key operations and the identity that performed them — this is crucial for detecting access failures or unauthorized key usage. Set up alerts for anomalous events: HSM key deletion requests, storage accounts falling back to Microsoft-managed keys unexpectedly, or access-denied errors in key vault logs.
- **Compliance Documentation:** Using HSM-backed keys in Azure Government aligns with **FedRAMP High**, **DoD IL4/IL5** controls requiring customer-managed and

hardware-protected keys. Maintain documentation of this configuration as **audit evidence**. For **DoD IL5** workloads, also consider whether additional constraints apply — IL5 workloads may require **Azure Government DoD regions** and potentially different handling, though IL4/IL5 capabilities for Key Vault are similar.

- **Testing & Rollback: Test this process on a non-production storage account first**. To **roll back** (remove CMK), reconfigure the storage account to use Microsoft-managed keys via Set-AzStorageAccount by clearing the Key Vault URI and Key Name, or via the Azure Portal. Azure transparently re-wraps the DEK with Microsoft keys — **there is no downtime**. Detach the user-assigned identity and optionally remove its HSM role if no longer needed.
- **Multi-Region / Disaster Recovery:** If storage accounts use **geo-redundant storage (GRS)**, ensure the secondary region can also access the encryption key. Managed HSM does not automatically replicate keys across regions. Either set up **two HSMs with identical keys** or ensure key availability via a soft-linked HSM accessible to both regions. In GCC High, regions are limited (e.g. **US Gov Virginia** and **US Gov Arizona**) — plan HSM and key placement to accommodate regional failover scenarios.
- **Resource Naming Conventions:** The script's naming convention (**key-<saname>-cr2-westus3**) ensures keys are **uniquely identifiable** and correlated to their storage accounts, aiding management and auditing within a shared HSM namespace. Verify that key names do not clash if multiple storage accounts with similar names from different subscriptions share the same HSM. The current scheme embeds the environment code (cr2) and region (westus3) in the key name to reduce collision risk.

Conclusion

This white paper presented a comprehensive, **script-driven methodology** for migrating Azure Storage Accounts to **customer-managed encryption keys stored in Azure Key Vault Managed HSM**. The PowerShell automation ensures each storage account receives a **dedicated RSA key** in the HSM (meeting compliance for hardware-backed key storage), **automates** the assignment of a managed identity and the configuration of encryption settings, and **logs all steps** for auditing and troubleshooting.

The migration places storage accounts under **full cryptographic control** of the organization — **only identities explicitly authorized can decrypt data at rest**. Microsoft's role is reduced to managing the HSM service and storage infrastructure without access to the customer's keys, aligning with **Zero Trust principles** and government cloud security requirements.

Organizations adopting this approach should:

1. **Review and customize** script variables (HSM name, identity, subscriptions) for their specific environment.
2. **Perform the migration in a test environment first**, then in production during a scheduled change window (the change is seamless, but coordination with change management is recommended).

3. **Implement continuous monitoring** on keys and storage accounts via Azure Monitor logs to catch issues early — for example, alerts on key vault access failures or storage encryption status changes.
4. **Plan for key rotations** — set a key rotation policy for the Managed HSM keys such that a new key version is generated periodically, and verify that storage accounts pick up the new version automatically.

By following these steps, Contoso and similarly regulated organizations will **significantly enhance the security of their data at rest** while maintaining full application functionality. The solution effectively ties together **Azure Storage encryption**, **Azure Managed HSM**, and **Azure AD identities** in a robust architecture suitable for **GCC High**, **Azure Government**, and **Commercial** cloud deployments.

This is a preliminary document and may be changed substantially prior to final commercial release of the software described herein.

The information contained in this document represents the current view of Microsoft Corporation on the issues discussed as of the date of publication. Because Microsoft must respond to changing market conditions, it should not be interpreted to be a commitment on the part of Microsoft, and Microsoft cannot guarantee the accuracy of any information presented after the date of publication.

This guide is for informational purposes only. Microsoft makes no warranties, express or implied, in this document.

Complying with all applicable copyright laws is the responsibility of the user. Without limiting the rights under copyright, no part of this document may be reproduced, stored in, or introduced into a retrieval system, or transmitted in any form or by any means (electronic, mechanical, photocopying, recording, or otherwise), or for any purpose, without the express written permission of Microsoft Corporation.

Microsoft may have patents, patent applications, trademarks, copyrights, or other intellectual property rights covering subject matter in this document. Except as expressly provided in any written license agreement from Microsoft, the furnishing of this document does not give you any license to these patents, trademarks, copyrights, or other intellectual property.

© 2020 Microsoft Corporation. All rights reserved.

The example companies, organizations, products, domain names, e-mail addresses, logos, people, places, and events depicted herein are fictitious. No association with any real company, organization, product, domain name, e-mail address, logo, person, place, or event is intended or should be inferred.

Microsoft, list Microsoft trademarks used in your white paper alphabetically are either registered trademarks or trademarks of Microsoft Corporation in the United States and/or other countries.

The names of actual companies and products mentioned herein may be the trademarks of their respective owners.